

LPII - Menu do Sistema e Cadastro Básico

1 – Instalando Softwares para Desenvolvimento e Testando Acesso ao Banco

Nesta disciplina é desenvolvida de forma incremental uma aplicação desktop, que executará no seu computador sem acesso à web. A aplicação terá uma interface gráfica e terá acesso ao banco de dados relacional MySQL para persistência dos dados informados pelos usuários.

Instale a versão mais atual do banco de dados relacional MySQL (MySQL Server 8.0).

- pesquise: MySQL Community Downloads
 - site: <https://dev.mysql.com/downloads/>
- baixe e instale: mysql-installer-web-community-8.0.28.0
- instale versão para desenvolvimento
 - customize a instalação desabilitando pacotes para Visual Studio
 - para ficar compatível com os tutoriais do LPII, forneça como senha: admin
- baixe conector java para JDBC (Connector/J 8.0.28)
 - baixe o executável: mysql-connector-java-8.0.28.jar
 - armazene em uma pasta por posterior vinculação no NetBeans

A seguir, será ilustrado um teste de acesso ao banco de dados relacional MySQL.

Passo 1 : Implantação e teste de conexão do NetBeans com o conector MySQL no NetBeans

- para acessar o banco de dados MySQL
 - Services – Drivers – MySQL (Connector/J driver)
 - mouse botão direito
 - Customize – Add
 - procure localização do conector Java: mysql-connector-java-8.0.28.jar
 - Connect using ...
 - informe a senha: admin
 - Test Connection

Passo 2 : Edite o script SQL de criação da tabela **Pessoas** e armazene um diretório **sql** (criar) no projeto

```
DROP TABLE IF EXISTS Pessoas;  
  
CREATE TABLE Pessoas (  
    CPF VARCHAR(15) NOT NULL PRIMARY KEY,  
    Nome VARCHAR(50) NOT NULL);
```

Passo 3 : Criação da Base de Dados para Teste

- [NetBeans](#) → [Serviços](#) → [Servidor MySQL](#) → [Conectar](#)
 - informe senha da instalação do MySQL se solicitado
- [Servidor MySQL](#) → [Criar Banco de Dados](#)
 - Novo Nome do Banco de Dados : teste
- [Servidor MySQL](#) → teste → [Conectar](#)
- [jdbc:mysql://localhost:3306/teste](#) → teste → [Tabelas](#) → [Executar Comando](#)
 - copiar e colar script de criação da tabela [Pessoas](#)
 - executar utilizando o ícone



Passo 4 : Crie o projeto TesteBD no NetBeans

- [NetBeans](#) → [Arquivo](#) → [Novo Projeto](#)
 - Escolher Projeto
 - Categorias : Java com Maven
 - Projetos : Aplicação Java
 - Nome e Localização
 - Nome do Projeto : TesteBD
 - Localização do Projeto : escolher um bom diretório para armazenar seus projetos
 - Criar Classe Principal : controle.TesteBD

Passo 5: Adicione o conector Java no arquivo de dependências do projeto (pom.xml)

- após a tag `</properties>` acrescente a dependência do conector mysql caracterizando a sua versão:

```
<dependencies>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.28</version>
  </dependency>
</dependencies>
```

Agora, você colocará o código do projeto TesteBD no NetBeans, para testar uma a inserção e a leitura em uma tabela no banco de dados.

Classe TesteBD : importação e atributos

```
package controle;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class TesteBD {
    static final String URL_BD = "jdbc:mysql://localhost:3306/teste";
    static final String USUÁRIO = "root";
    static final String SENHA = "admin";
    private static Connection conexão = null;
    private static Statement comando = null;
```

Classe TesteBD : métodos para criar e fechar conexão e comando

```
public static void criaConexãoComando () {
    try {
        conexão = DriverManager.getConnection (URL_BD, USUÁRIO, SENHA);
        comando = conexão.createStatement ();
    } catch (SQLException exceção_sql) {
        System.out.println("Erro na criação da conexão");
        exceção_sql.printStackTrace ();
    }

    public static void fechaComandoConexão () {
        try {
            comando.close();
            conexão.close();
        } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    }
}
```

Classe TesteBD : método main

- removendo registros anteriores da tabela Pessoas
- inserindo um registro na tabela com nome e CPF
- consultando nome a partir do CPF e imprimindo o nome consultado

```
public static void main(String[] args) {
    criaConexãoComando ();
    String sql;
    sql = "DELETE FROM Pessoas";
    try {
        comando.executeUpdate(sql);
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    sql = "INSERT INTO Pessoas (CPF, Nome) VALUES ('111.111.111-11', 'Ana Julia')";
    try {
        comando.executeUpdate(sql);
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    sql = "SELECT Nome FROM Pessoas WHERE CPF = '111.111.111-11'";
    ResultSet lista_resultados = null;
    try {
        lista_resultados = comando.executeQuery(sql);
        while (lista_resultados.next()) {
            System.out.println ("Nome: " + lista_resultados.getString("Nome"));
        }
        lista_resultados.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    fechaComandoConexão ();
}
}
```

O resultado gerado deverá ser:

```
Nome: Ana Julia
```

2 - Estruturação do Projeto de Referência do Tutorial

No projeto de referência deste tutorial, são criados quatro pacotes com as seguintes classes:

- persistência : com a classe BD, que disponibiliza os métodos para criar e remover a conexão com o banco de dados;
- entidades : com as classes Amigo e Ator, correspondentes às entidades sem referências;
- interfaces (a palavra interface é uma palavra-chave em Java e não pode ser utilizada) : com as classes JanelaAmigosCinema, JanelaCadastroAmigos e JanelaCadastroAtores;
- controles : com as classes ControladorCadastroAmigos e ControladorCadastroAtores.

Cada controlador recebe solicitações e informações, que usuário fornece a partir de uma janela de cadastro de uma dada entidade, e interage com a respectiva entidade para verificar a viabilidade de inserir, alterar ou remover, e para solicitar a efetivação da respectiva operação no banco de dados.

As demais janelas, serão ilustradas nos próximos três tutoriais: cadastro de filmes, cadastro das subclasses de filmes, cadastro e pesquisa de avaliações.

Neste Tutorial, todos os atributos das entidades são lidos como textos. No Tutorial 2 serão ilustrados: (a) componentes adicionais utilizados para representar valores de enumerados (ex: estado_civil), seleção de item do tipo boolean (ex: ganhador_oscar) e os objetos com relacionamentos múltiplos. No Tutorial 3, será possível representar atributos de subclasses. No Tutorial 4, será ilustrado a pesquisa de avaliações, no banco de dados, a partir de atributos das entidades: Amigo, Filme e Avaliação.

3 - Conexão com o banco de dados e Script de criação das Tabelas

Para persistir os dados, informados pelos usuários, é necessária a criação (e posterior remoção) de uma conexão com a base de dados. A classe BD, do pacote persistência, provê essa funcionalidade:

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class BD {

    static final String URL_BD = "jdbc:mysql://localhost/amigos";
    static final String USUÁRIO = "root";
    static final String SENHA = "admin";
    public static Connection conexão = null;

    public static void criaConexão () {
        try {
            conexão = DriverManager.getConnection (URL_BD, USUÁRIO, SENHA);
        } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    }

    public static void fechaConexão () {
        try {
            conexão.close();
        } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    }
}
```

Os comandos SQL utilizados na ilustração da aplicação desktop, são parametrizados (ver seção 5) e não podem ser criados a priori, como foi ilustrado na seção 1 para a aplicação TesteBD.

No subdiretório sql no diretório do projeto, é definido o script de criação das tabelas Amigos e Atores.

```
DROP TABLE IF EXISTS Amigos;
DROP TABLE IF EXISTS Atores;

CREATE TABLE Amigos (
    Nome VARCHAR(50) NOT NULL PRIMARY KEY,
    Apelido VARCHAR(30),
    Cidade VARCHAR(50) NOT NULL,
    Email VARCHAR(50) NOT NULL,
    Sexo INT NOT NULL,
    EstadoCivil INT NOT NULL,
    Whatsapp VARCHAR(30)) ;

CREATE TABLE Atores (
    Nome VARCHAR(50) NOT NULL PRIMARY KEY,
    AnoNascimento INT NOT NULL,
    Sexo VARCHAR(9) NOT NULL,
    GanhadorOscar BIT(1) NOT NULL);
```

De forma semelhante à utilizada na seção 1, crie o banco de dados **amigos** e execute o script para criar a tabela **Amigos** nesse banco.

4 – Construindo a JanelaAmigosCinema no pacote interfaces

A aplicação desktop, ilustrada neste tutorial, é intitulada Clube de Amigos do Cinema, em referência a amigos interessados em compartilhar informações sobre filmes preferidos. Essa aplicação suporta o cadastro de amigos, atores, filmes e avaliações (sobre filmes assistidos por alguém do clube) e a pesquisa de avaliações cadastradas por vários filtros opcionais, ou seja, o usuário pode selecionar os valores dos filtros de interesse, do conjunto de filtros disponibilizados para pesquisa.

Nesta seção é ilustrada a construção da JanelaAmigosCinema, janela principal da aplicação desktop, com os menus utilizados para visualizar as demais janelas da aplicação.

Criando o projeto AmigosCinema no NetBeans:

- **Arquivo → Novo Projeto**
 - **Escolher Projeto**
 - **Categorias** : selecione **Java**
 - **Projetos** : selecione **Aplicação Java**
 - **Nome e Localização**
 - **Nome do Projeto** : **LPI-E1**
 - **Localização do Projeto** : escolha um diretório para armazenar seu projeto
 - **Criar Classe Principal** : não assinalar

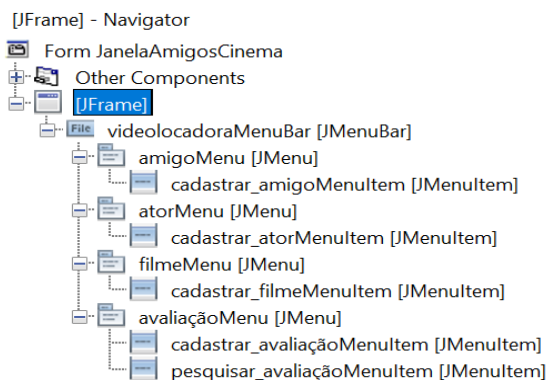
Criando a janela principal da aplicação desktop **JanelaAmigosCinema**:

- **AmigosCinema** → **Novo** → **Form JFrame**
 - **Nome e Localização**
 - **Nome da Classe** : **JanelaAmigosCinema**
 - **Pacote** : **interfaces**

As janelas são containers de componentes gráficos que herdam as características básicas (título, ícones comuns) da classe **JFrame**. A maioria dos componentes gráficos do pacote **javax.swing** inicia com um **J**, que não traz nenhuma informação que distinga os componentes. Portanto, serão referenciados, nos tutoriais desta disciplina, sem o **J** inicial (neste caso: **Frame**). As janelas são representadas no NetBeans em duas abas principais:

- **Código-Fonte**
- **Projeto** : que contém o esboço do layout da janela

Agora vamos apreender como criar os menus da janela principal da aplicação. Na realidade serão utilizados 3 tipos de componentes: barra de menus (**MenuBar**), no qual são associados menus (**Menu**), no qual são associados itens de menu (**MenuItem**). Observe na aba Navegador (se não estiver visível, selecione no NetBeans: **Janela** → **Navegador**), a hierarquia dos componentes que será criada para representar os menus na janela principal:



Quando a aba **Projeto** é selecionada, o NetBeans disponibiliza uma área central para visualização do esboço do layout da janela, e duas abas (localizadas à direita) para edição do layout da janela:

- **Paleta** : disponibiliza os componentes que serão arrastados para a área central para compor o layout da janela agrupados em conjuntos de componentes afins
- **Propriedades** : disponibiliza abas internas que serão utilizadas para configurar as propriedades do componente inserido no layout
 - **Propriedades** : para alterar propriedades do componente (ex: título da janela)
 - **Eventos** : para associar um tratador de evento a um dado componente (ex: método `cadastrarAmigo` a ser chamado quando o item de menu **Cadastrar**, vinculado ao menu **Amigo** é selecionado)
 - **Código**: nome da variável que será associada ao componente sendo inserido na janela
 - suponha que você criou o item de menu Cadastrar vinculado ao menu Amigo (vinculado à barra de menus)
 - o Netbeans vai disponibilizar um nome de variável pouco significativo
 - `jMenuItem1`
 - altere para um nome que mantenha o tipo do componente, agregue a funcionalidade do componente e remova os caracteres irrelevantes (`j` e `1`)
 - `cadastrar_amigoMenuItem`

Para a criação do menus da janela principal é mais cômodo utilizar a aba **Navegador** para visualizar a hierarquia dos componentes sendo criados. Vamos começar criando a barra de menus `amigos_cinemaMenuBar`:

- **Navegador** → botão direito do mouse em **Frame** → **Adicionar da Paleta** → **Menus Swing** → **Barra de Menu**
 - será inserido como filho de **Frame**
 - `jMenuBar1`
 - aba **Propriedades** → **Código**
 - **Nome da Variável** : altere de `jMenuBar1` para `amigos_cinemaMenuBar`

Para criar o menu `amigoMenu`:

- **Navegador** → botão direito do mouse em `amigos_cinemaMenuBar` → **Adicionar Menu**
 - alguns menus serão inseridos como filhos de `amigos_cinemaMenuBar`
 - remomeie cada menu para: **Amigo**, **Filme** e **Avaliação** (acrescente ou remova menus se necessário)
 - exemplificando com o menu **Amigo**
 - `jMenuItem`
 - aba **Propriedades** → **Código**
 - **Nome da Variável** : `amigoMenu`
 - aba **Propriedades** → **Propriedades**
 - **text** : **Amigo**

Para criar o item de menu `cadaststrar_amigoItemMenu`:

- Navegador → botão direito do mouse em `amigoMenu` → Adicionar da Paleta → Item de Menu
 - um item de menu será inserido como filho de `amigoMenu`
 - `jItemMenu1`
 - aba `Propriedades` → `Código`
 - Nome da Variável : `cadaststrar_amigoItemMenu`
 - aba `Propriedades` → `Propriedades`
 - `text` : `Cadastrar`
 - aba `Propriedades` → `Eventos`
 - `actionPerformed` : `cadaststrarAmigo`

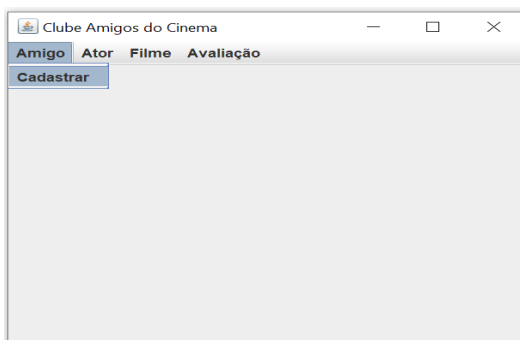
O método tratador de evento `cadaststrarAmigo` será criado no código fonte da janela em construção para que você posteriormente implementar o seu corpo. Quando o usuário gerar um evento na janela principal da aplicação, selecionando o item de menu `Cadastrar` associado ao menu `Amigo`, o método tratador de evento `cadaststrarAmigo` será chamado. Seu corpo é implementado com a criação de um objeto da classe `ControladorCadastroAmigos` que por sua vez irá criar e tornar visível a `JanelaCadastroAmigos` (seção 3).

```
private void cadaststrarAmigo(java.awt.event.ActionEvent evt) {
    new ControladorCadastroAmigos();
}
```

O construtor da classe `ControladorCadastroAmigos`, do pacote `controles`, cria um objeto da `JanelaCadastroAmigos` (seção 3), repassando o seu objeto (`this`) para que a objeto da janela possa lhe repassar solicitações dos usuários, e ativando a visibilidade da janela.

```
public ControladorCadastroAmigos() {
    new JanelaCadastroAmigos(this).setVisible(true);
}
```

Observe a ilustração da seleção do item de menu `Cadastrar` do menu `Amigo` na `JanelaAmigosCinema`:



O código relacionado com os componentes gráficos criados, bem como suas alterações de propriedades (realizadas a partir da aba `Propriedades` → `Código`), são inseridos no método `initComponents`, que criado diretamente pelo NetBeans e não pode ser editado pelo implementador da aplicação (delimitado por uma área em azul). As variáveis correspondentes aos componentes criados são inseridas ao final do código fonte da janela em uma área azul que também não pode ser editada diretamente, somente sendo removidas ou alteradas a partir da aba `Propriedades` → `Código`.

As assinaturas (tipo de retorno, nome do método e parâmetros) de métodos criados como tratadores de eventos, também só podem ser removidas ou alteradas a partir da aba [Propriedades](#) → [Eventos](#).

Alterando propriedades na [JanelaAmigosCinema](#):

- selecione a janela [Frame](#) na aba [Navegador](#) (ou na área do esboço do layout)
- [Propriedades](#) → [Propriedades](#)
 - [defaultCloseOperation](#) : [DISPOSE](#)
 - [title](#) : Clube de Amigos do Cinema
 - [preferredSize](#) : [400, 300]
- [Propriedades](#) → [Eventos](#)
 - [windowClosed](#) : [terminarSistema](#)

O tratador de evento [terminarSistema](#) é associado somente à janela principal da aplicação, para que quando for selecionado o ícone de fechamento da janela (X), além de remoção da visualização da janela, como ocorre com as demais janelas da aplicação sejam, adicionalmente, fechada a conexão com o banco de dados e encerrada a execução da aplicação. A seguir, é ilustrado o seu código:

```
private void terminarSistema(java.awt.event.WindowEvent evt) {  
    BD.fechaConexão();  
    System.exit(0);  
}
```

No construtor da [JanelaAmigosCinema](#) é criada a conexão com o banco de dados:

```
public JanelaAmigosCinema() {  
    BD.criaConexão();  
    initComponents();  
}
```

Para cada um dos itens de menu, associados aos menus Filme e Avaliação, serão associados tratadores de eventos que provisoriamente serão implementados para mostrar a informação de serviço indisponível. Por exemplo, a configuração do tratador de eventos para o item de menu [Pesquisar](#) do menu [Avaliação](#):

- selecione o item de menu [pesquisar_avaliacoesItemMenu](#) na aba [Navegador](#) (ou na área do esboço do layout)
- [Propriedades](#) → [Eventos](#)
 - [actionPerformed](#) : [pesquisarAvaliações](#)

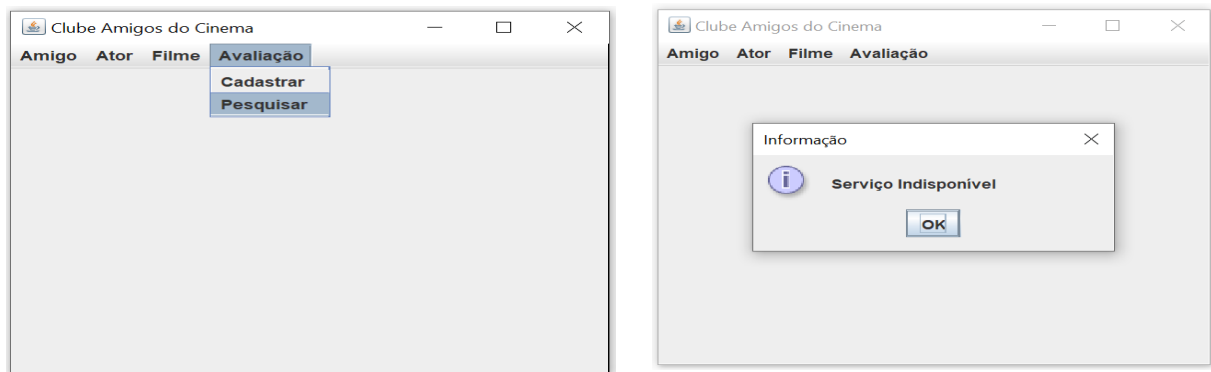
Os códigos do tratador de eventos [pesquisarAvaliações](#), e do método auxiliar [informarServiçoIndisponível](#), são ilustrados a seguir:

```
private void informarServiçoIndisponível() {  
    JOptionPane.showMessageDialog (this, "Serviço Indisponível", "Informação",  
        JOptionPane.INFORMATION_MESSAGE);  
}  
  
private void pesquisarAvaliação(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

O método `informarServiçoIndisponível`, utiliza um método estático de `OptionPane` (omitindo o `J` inicial) que recebe os seguintes argumentos:

- objeto da janela mãe da janela auxiliar : `this` (objeto da `JanelaAmigosCinema`);
- string da mensagem a ser informada : "Serviço Indisponível";
- título da janela auxiliar : "Informação"
- tipo da mensagem a ser informada pela janela auxiliar :
 - `JOptionPane.INFORMATION_MESSAGE`

Observe a ilustração da seleção do item de menu `Pesquisar` do menu `Avaliação`, seguida da ilustração da informação de `Serviço Indisponível` na janela auxiliar:



5 – Construindo a `JanelaCadastroAmigos` e as demais classes envolvidas

Neste Tutorial é ilustrado o cadastro básico da entidade sem referências `Amigo`. A implementação do cadastro da entidade sem referências `Ator` é semelhante a da classe `Amigo`. Portanto, para a entidade `Ator` são ilustrados (seção 6) somente: (a) os enumerados e os atributos da classe `Ator`; e (b) a imagem da interface da `JanelaCadastroAtores`.

A classe `Amigo`, do pacote `entidades`, é definida com os seguintes enumerados e atributos:

```
public class Amigo {

    public enum Sexo { masculino, feminino };
    public enum EstadoCivil { solteiro, casado, divorciado, viúvo};

    private String nome, apelido, cidade, email, whatsapp;
    private Sexo sexo;
    private EstadoCivil estadoCivil;
```

Na [JanelaCadastrarAmigos](#), a representação do formulário de preenchimento dos atributos, com os dados preenchidos, é a seguinte:

The screenshot shows a window titled "Cadastrar Amigos". At the top, there is a dropdown menu labeled "Amigos Cadastrados" with the selected value "Francisco Silveira [Chico Bento]". Below this, there are several text input fields: "Nome" (filled with "Francisco Silveira"), "Apelido" (filled with "Chico Bento"), "Sexo" (filled with "M"), "Estado Civil" (filled with "C"), "Cidade" (filled with "Dourados"), "Email" (filled with "francisco.silveira@gmail.com"), and "Whatsapp" (filled with "67-92222-2222"). At the bottom of the form, there are five buttons: "Inserir", "Consultar", "Alterar", "Remover", and "Limpar".

A configuração da JanelaCadastroAmigos é a seguinte:

- [Propriedades](#) → [Propriedades](#)
 - [defaultCloseOperation](#) : [DISPOSE](#)
 - [title](#) : [Cadastrar Amigos](#)
 - [preferredSize](#) : [\[567, 313\]](#)

Para a visualização dos rótulos associados ao campos a serem preenchidos é utilizado o componente [Label](#). Ilustrando a configuração das propriedades do rótulo do nome:

- arraste [Label](#) da [Paleta](#) → [Controles Swing](#) para o esboço do layout (ou utilize o [Navegador](#) para inserir como filho do [Frame](#))
- [Propriedades](#) → [Código](#)
 - [Nome da Variável](#) : [nomeLabel](#)
- [Propriedades](#) → [Propriedades](#)
 - [text](#) : [Nome](#)

A cada objeto inserido na [JanelaCadastroAmigos](#) algumas informações do objeto são mostradas no componente [ComboBox](#) da janela. Ilustrando a configuração de suas propriedades:

- arraste [ComboBox](#) da [Paleta](#) → [Controles Swing](#) para o esboço do layout (ou utilize o [Navegador](#) para inserir como filho do [Frame](#))
- [Propriedades](#) → [Código](#)
 - [Nome da Variável](#) : [amigos_cadastradosComboBox](#)
 - [TypeParameters](#) : remova a palavra [String](#)
- [Propriedades](#) → [Propriedades](#)
 - [maximumRowCount](#) : [8](#)
 - [model](#) : [selecione ...](#)
 - no [model property](#) selecione : [Custom code](#)
 - preencha [setModel](#) com : [new DefaultComboBoxModel \(amigos_cadastrados\)](#)

Conforme ilustrado e explicado na página 15, a variável `amigos_cadastrados` deve ser definida no construtor da `JanelaCadastroAmigos`.

Para o preenchimento dos campos é utilizado o componente `TextField`. Ilustrando a configuração das propriedades do campo de texto do nome:

- arraste `Campo de Texto` da `Paleta` → `Controles Swing` para o esboço do layout (ou utilize o `Navegador` para inserir como filho do `Frame`)
- `Propriedades` → `Código`
 - Nome da Variável : `nomeTextField`
- `Propriedades` → `Propriedades`
 - `columns` : 50
 - `text` : string vazio
 - `preferredSize` : [456, 20]

Crie um par `Label` e `Campo de Texto` para cada um desses dados da entidade Amigo, com os seguintes rótulos: `Apelido`, `Cidade`, `Email`, `Sexo`, `Estado Civil` e `Whatsapp`.

Os botões de comando devem encapsalados por um painel de container (`Panel`), para pode ser posicionados com um grupo no layout da janela. Ilustrando a configuração das propriedades do painel:

- arraste `Painel` da `Paleta` → `Contêineres` para o esboço do layout (ou utilize o `Navegador` para inserir como filho do `Frame`)
- `Propriedades` → `Código`
 - Nome da Variável : `comandosPanel`
- `Propriedades` → `Propriedades`
 - `border` : `Borda de Linha`
 - para visualizar o painel : após inserir todos os comandos você poderá alterar para
 - `border` : `Sem Borda`
- `Navegador`
 - botão direito → `Definir Layout` : `Layout de Fluxo` (layout default do painel)

Insira o botões de comando como filhos do `comandosPanel` no `Navegador`. Ilustrando com o comando para inserção:

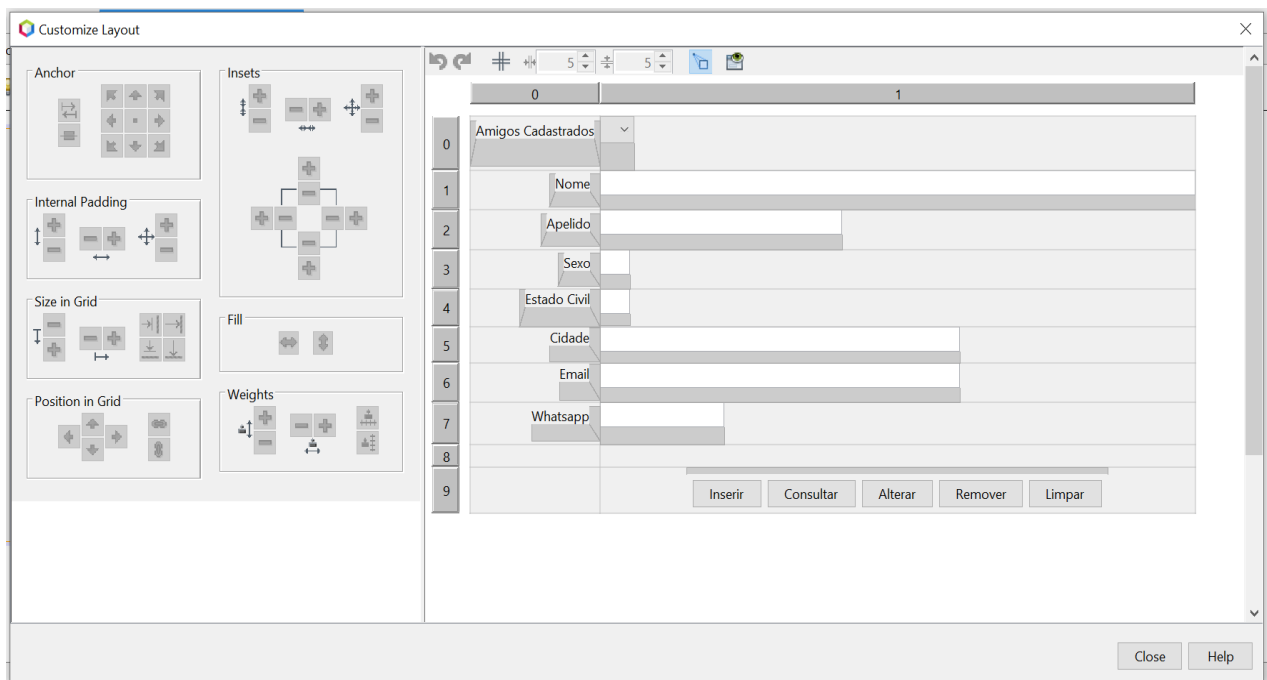
- arraste `Botão` da `Paleta` → `Controles Swing` para o esboço do layout (ou utilize o `Navegador`, para inserir como filho do `comandosPanel`)
- `Propriedades` → `Código`
 - Nome da Variável : `inserirButton`
- `Propriedades` → `Propriedades`
 - `text` : `Inserir`

Repita a mesma operação para os botões de comando com os textos: `Consultar`, `Alterar`, `Remover` e `Limpar`.

Existem várias opções de layout. O layout default do Frame é o Layout da Borda (Border Layout). Esse layout é muito limitado, permitindo definir a área da janela somente em cinco subáreas: centro, sul, norte, leste e oeste. Para ter um controle maior sobre a localização dos componentes na janela vamos utilizar o [Layout de Conjunto de Grades \(GridBagLayout\)](#), que é o layout mais customizável disponível:

- no [Navegador](#) selecione [Frame](#) → botão direito do mouse → [Definir Layout](#) → [Layout de Conjunto de Grades](#)

Observe também o distanciamento entre os rótulos e os campos de texto, e entre os pares rótulo e campo de texto e os seus pares acima e abaixo, bem como o distanciamento do grupo de botões de comando. Organizar o layout dessa forma é bem simples utilizando o [GridBagLayout](#). Clique na área do esboço do layout da janela e selecione, com o botão direito do mouse: [Personalizar Layout](#). Você vai visualizar a janela [Personalizar Layout](#):



Essa janela tem duas regiões. Na região direita você tem os componentes gráficos posicionados em uma matriz com 2 colunas e 10 linhas. Na região esquerda você tem as funcionalidades que podem ser aplicadas a um componente ou a um grupo de componentes. Trabalhar com um grupo de componentes de uma dada coluna ou de uma dada linha facilita a organização do layout. As opções mais úteis são:

- ancorar uma coluna de componentes à esquerda ou à direita
 - selecionar os rótulos da coluna 0, e ancorá-los ([Ancorar](#)) à direita (de forma que a extremidade direita de todos eles esteja alinhada na vertical)
 - selecionar os componentes à direita dos rótulos da coluna 1, e ancorá-los ([Ancorar](#)) à esquerda (de forma que a extremidade esquerda de todos eles esteja alinhada na vertical)
- ajustar a distância de uma coluna dos componentes de uma dada coluna à direita ou à esquerda
 - selecionar os rótulos da coluna 0, e a ajustar a distância dos componentes da coluna 1 ([Insets](#))
- ajustar a distância dos componentes de uma dada linha acima ou abaixo
 - selecionar os componentes em uma dada linha (um par rótulo e seu componente associado, ou o painel de botões de comando) e ajustar a distância dos componentes da linha de cima e/ou da linha de baixo ([Insets](#)).

Caso os componentes não estejam posicionados inicialmente na linha ou na coluna que você deseja, você pode arrastá-los e posicionar na linha e coluna desejada. Também pode inserir uma nova linha acima ou abaixo de uma linha existente, para posicionar um componente. Idem com as colunas.

Para complementar essa seção, serão ilustrados (em subseções) os métodos da [JanelaCadastroAmigos](#). Vamos visualizar as importações, uma função auxiliar (método estático) de conversão de string para elemento de enumerado e o construtor da janela. O método construtor recebe o objeto do controlador, para armazená-lo e utilizá-lo quando um botão de comando for selecionado pelo usuário e tiver que repassar informações para o controlador decidir sobre a realização do comando solicitado.

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroAmigos;
import entidades.Amigo;
import entidades.Amigo.Sexo;
import entidades.Amigo.EstadoCivil;
import javax.swing.DefaultComboBoxModel;

public class JanelaCadastroAmigos extends javax.swing.JFrame {

    public static EstadoCivil converteStringParaEstadoCivil(String estado_civil_str) {
        switch(estado_civil_str){
            case "C": return EstadoCivil.casado;
            case "D": return EstadoCivil.divorciado;
            case "S": return EstadoCivil.solteiro;
            case "V": return EstadoCivil.viúvo;
            default: return null;
        }
    }

    ControladorCadastroAmigos controlador;
    Amigo[] amigos_cadastrados;

    public JanelaCadastroAmigos(ControladorCadastroAmigos controlador) {
        this.controlador = controlador;
        amigos_cadastrados = Amigo.getVisões();
        initComponents();
        limparCampos(null);
    }
}
```

A função (método estático) [getVisões](#), da classe [Amigo](#) retorna um conjunto de objetos reduzidos para serem mostrados no componente [ComboBox](#), utilizado nesta interface para representar os objetos cadastrados da classe [Amigo](#). Neste componente são mostrados somente os atributos nome e apelido da classe [Amigo](#).

```
public static Amigo[] getVisões() {
    String sql = "SELECT Nome, Apelido FROM Amigos";
    ResultSet lista_resultados = null;
    ArrayList<Amigo> visões = new ArrayList();
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            String nome = lista_resultados.getString("Nome");
            String apelido = lista_resultados.getString("Apelido");
            visões.add(new Amigo (nome, apelido));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return visões.toArray(new Amigo[visões.size()]);
}
```

O método `limparCampos`, ilustrado na seção 5.5, é criado como tratador de eventos para ser executado quando o usuário clica no botão `Limpar`. Por esse motivo esse método é definido pelo NetBeans com parâmetro `evt` para receber o evento como argumento. No entanto, esse método também pode ser chamado sem que nenhum evento tenha sido gerado; basta passar `null` como argumento para o parâmetro `evt`.

5.1 - O Tratamento de Eventos para Inserir um Amigo

O tratador de eventos `inserirAmigo`, associado ao botão de comando `Inserir`, solicita a leitura dos campos informados na interface ao método `obterAmigoInformado`, que retorna um objeto da classe `Amigo`. Se todos os campos forem lidos corretamente, o objeto retornado não é nulo e, portanto, é repassado ao objeto da classe `ControladorCadastroAmigos`, que checa se a inserção é possível e coordena a sua execução no banco de dados. Caso não retorne mensagem de erro (em caso de insucesso), é obtida uma visão da classe `Amigo` (objeto reduzido somente com os atributos `nome` e `apelido`) que é inserida no componente `ComboBox` (que mostra os amigos cadastrados) e o último objeto cadastrado é selecionado para ser mostrado no `ComboBox`.

```
private void inserirAmigo(java.awt.event.ActionEvent evt) {
    Amigo amigo = obterAmigoInformado();
    String mensagem_erro = null;
    if (amigo != null) mensagem_erro = controlador.inserirAmigo(amigo);
    else mensagem_erro = "Algum atributo do amigo não foi informado";
    if (mensagem_erro == null) {
        Amigo visão = amigo.getVisão();
        amigos_cadastradosComboBox.addItem(visão);
        amigos_cadastradosComboBox.setSelectedItem(visão);
    } else informarErro (mensagem_erro);
}
```

O método `getVisão` da classe `Amigo`, cria um objeto com atributos reduzidos para ser exibido no componente `ComboBox`, a partir de um construtor que recebe somente os atributos `nome` e `apelido` como argumentos.

```
public Amigo(String nome, String apelido) {
    this.nome = nome;
    this.apelido = apelido;
}

public Amigo getVisão() { return new Amigo (nome, apelido); }
```

O método auxiliar `obterAmigoInformado`, obtém todas as informações preenchidas nos campos de texto do formulário de cadastro, e retorna o objeto da entidade `Amigo`, construído a partir das informações do usuário. Cada campo de texto é testado, para verificar se não está vazio. Como todos os campos são obrigatórios, qualquer campo vazio resultará no retorno de `null` em lugar do objeto.

Para converter um string (com a letra inicial de um estado civil) para o correspondente elemento do enumerado `EstadoCivil` (da classe `Amigo`), é utilizado um método estático auxiliar. No caso do enumerado `Sexo` (da classe `Amigo`), foi implementado diretamente (no método `ObterAmigoInformado`) porque o enumerado tem menos elementos.

```
private Amigo obterAmigoInformado() {
    String nome = nomeTextField.getText();
    if (nome.isEmpty()) return null;
    String apelido = apelidoTextField.getText();
    if (apelido.isEmpty()) apelido = null;
    String cidade = cidadeTextField.getText();
    if (cidade.isEmpty()) return null;
    String email = emailTextField.getText();
    if (email.isEmpty()) return null;
    String sexo_str = sexoTextField.getText();
    Sexo sexo = Sexo.feminino;
    if (sexo_str.equals("M")) sexo = Sexo.masculino;
    else if (!sexo_str.equals("F")) return null;
    String estado_civil_str = estado_civilTextField.getText();
    if (estado_civil_str.isEmpty()) return null;
    EstadoCivil estado_civil = converteStringParaEstadoCivil(estado_civil_str);
    if (estado_civil == null) return null;
    String whatsapp = whatsappTextField.getText();
    if (whatsapp.isEmpty()) whatsapp = null;
    return new Amigo(nome, apelido, cidade, email, sexo, estado_civil, whatsapp);
}
```

O método auxiliar utilizado para informar erro na operação é ilustrado abaixo:

```
private void informarErro (String mensagem) {
    JOptionPane.showMessageDialog (this, mensagem, "Erro", JOptionPane.ERROR_MESSAGE);
}
```

Todos os métodos do controlador retornam uma mensagem de erro, caracterizando o insucesso ocorrido, ou retornam um string nulo, para sinalizar sucesso na execução da operação. A seguir a ilustração do método `inserirAmigo` da classe `ControladorCadastroAmigos`, do pacote `controles`:

```
public String inserirAmigo (Amigo amigo) {
    Amigo amigol = Amigo.buscarAmigo(amigo.getNome());
    if (amigol == null) return Amigo.inserirAmigo(amigo);
    else return "Nome de amigo já cadastrado";
}
```

Embora essa implementação seja simples, não conclua apressadamente que as implementações dos métodos dos controladores, por serem simples, possam ser incorporadas nas implementações das janelas da interface ou nas entidades. São os controladores que decidem sobre as regras de negócios da aplicação.

Em uma biblioteca, por exemplo, o empréstimo de uma obra, envolve várias consistências: (a) se o consulente (usuário da biblioteca) está cadastrado; (b) se obra está cadastrada; (c) se existe um exemplar da obra disponível ou reservado para o consulente.

Após essas verificações, será necessário criar um empréstimo e remover a reserva para o consulente, quando for o caso. Observe que as interfaces só devem tratar de entrada e saída e as entidades só devem tratar dos seus dados. Portanto, a intermediação do controlador que acessa várias entidades para decidir sobre a viabilidade da operação é fundamental para isolar as funcionalidades de entrada e saídas das funcionalidades de manipulação dos dados das entidades. Imagine agora que você está implementando uma seguradora. As regras de negócio da seguradora são mais complexas e podem envolver a consistência de vários dados para decidir sobre a viabilidade da operação, bem como, a geração dos cálculos do seguro.

Os métodos de acesso à base de dados são definidos como estáticos (chamados diretamente pelo nome da classe) na classe entidade Amigo. A seguir a ilustração do método estático [buscarAmigo](#) da entidade [Amigo](#), do pacote [entidades](#):

```
public static Amigo buscarAmigo (String nome) {
    String sql = "SELECT * FROM Amigos WHERE Nome = ?";
    ResultSet lista_resultados = null;
    Amigo amigo = null;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, nome);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            amigo = new Amigo (nome,
                lista_resultados.getString("Apelido"),
                lista_resultados.getString("Cidade"),
                lista_resultados.getString("Email"),
                Sexo.values()[lista_resultados.getInt("Sexo")],
                EstadoCivil.values()[lista_resultados.getInt("EstadoCivil")],
                lista_resultados.getString("Whatsapp"));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace();
        amigo = null;
    }
    return amigo;
}
```

A seguir a ilustração do método estático [inserirAmigo](#) da entidade [Amigo](#), do pacote [entidades](#):

```
public static String inserirAmigo(Amigo amigo) {
    String sql = "INSERT INTO Amigos (Nome, Apelido, Cidade, Email, Sexo, EstadoCivil, "
        + " Whatsapp) VALUES (?, ?, ?, ?, ?, ?, ?)";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, amigo.getNome());
        comando.setString(2, amigo.getApelido());
        comando.setString(3, amigo.getCidade());
        comando.setString(4, amigo.getEmail());
        comando.setInt(5, (amigo.getSexo().ordinal()));
        comando.setInt(6, amigo.getEstadoCivil().ordinal());
        comando.setString(7, amigo.getWhatsapp());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace();
        return "Erro na Inserção do Amigo no BD";
    }
}
```

5.2 - O Tratamento de Eventos para Consultar um Amigo

Na implementação do tratador de eventos `consultarAmigo` associado ao botão de comando `Consultar`, a chave do amigo (`nome`) é lida no campo de texto rotulado por `Nome`. A partir da chave, o objeto da entidade `Amigo` é buscado diretamente a partir de um método estático da classe `Amigo`. Para consultas o controlador não é utilizado, porque não há necessidade de tomar nenhuma decisão a respeito da viabilidade da execução, como ocorre em situações de inserção, alteração ou remoção. Todos os métodos de acesso ao banco de dados são definidos como métodos estáticos da entidade associada à informação que será consultada, inserida, alterada ou removida no banco de dados.

Observe que a utilização de `nome` como palavra-chave é aceitável numa aplicação restrita como um `Clube de Amigos do Cinema`, onde a quantidade de amigos é presumivelmente reduzida, mas não é adequado para um sistema que cadastre clientes ou pacientes, por exemplo. Nestes casos o `CPF` é a chave mais adequada porque podem existir pessoas com o mesmo nome.

```
private void consultarAmigo(java.awt.event.ActionEvent evt) {
    Amigo visão = (Amigo) amigos_cadastradosComboBox.getSelectedItem ();
    Amigo amigo = null;
    String mensagem_erro = null;
    if (visão != null) {
        amigo = Amigo.buscarAmigo (visão.getNome());
        if (amigo == null) mensagem_erro = "Amigo não cadastrado";
    } else mensagem_erro = "Nenhum amigo selecionado";
    if (mensagem_erro == null) {
        nomeTextField.setText(amigo.getNome());
        String apelido = amigo.getApelido();
        if (apelido == null) apelido = "";
        apelidoTextField.setText(apelido);
        cidadeTextField.setText(amigo.getCidade());
        emailTextField.setText(amigo.getEmail());
        String whatsapp = amigo.getWhatsapp();
        if (whatsapp == null) whatsapp = "";
        whatsappTextField.setText(whatsapp);
        sexoTextField.setText(((amigo.getSexo() + "").toUpperCase().charAt(0) + ""));
        estado_civilTextField.setText(((amigo.getEstadoCivil() + "").toUpperCase().charAt(0)
            + ""));
    } else informarErro (mensagem_erro);
}
```

5.3 - O Tratamento de Eventos para Alterar um Amigo

A implementação do tratador de eventos `alterarAmigo` associado ao botão de comando `Alterar`, é semelhante à implementação do tratador de eventos `inserirAmigo`.

```
private void alterarAmigo(java.awt.event.ActionEvent evt) {
    Amigo amigo = obterAmigoInformado();
    String mensagem_erro = null;
    if (amigo != null) mensagem_erro = controlador.alterarAmigo(amigo);
    else mensagem_erro = "Algum atributo do amigo não foi informado";
    if (mensagem_erro == null) {
        Amigo visão = getVisãoAlterada(amigo.getNome());
        if (visão != null) {
            visão.setApelido(amigo.getApelido());
            amigos_cadastradosComboBox.updateUI();
            amigos_cadastradosComboBox.setSelectedItem(visão);
        }
    } else informarErro (mensagem_erro);
}
```

Para alterar um amigo, é necessário informar todos os dados do mesmo, podendo alterar qualquer atributo, menos a sua chave (**nome**). O usuário tem duas possibilidades: (a) consultar o amigo de interesse e alterar os atributos que deseja; ou (b) preencher todos os campos do formulário. Se optar pela segunda opção, o usuário poderá informar a chave do usuário, independentemente do usuário selecionado no **usuários_cadastradosComboBox**. Para prevenir essa situação, é necessário procurar a visão cadastrada que corresponde à chave informada pelo usuário, para atualizar os demais atributos informados na visão. Para suportar essa funcionalidade, é utilizado o método auxiliar privado **getVisãoAlterada**:

```
private Amigo getVisãoAlterada(String nome) {
    for (Amigo visão : amigos_cadastrados) {
        if (visão.getNome().equals(nome)) return visão;
    }
    return null;
}
```

A seguir, a ilustração do método **alterarAmigo** da classe **ControladorCadastroAmigos**, do pacote **controles**:

```
public String alterarAmigo(Amigo amigo) {
    Amigo amigo1 = Amigo.buscarAmigo(amigo.getNome());
    if (amigo1 != null) return Amigo.alterarAmigo(amigo);
    else return "Nome de amigo não cadastrado";
}
```

A seguir, a ilustração do método estático **alterarAmigo** da entidade **Amigo**, do pacote **entidades**:

```
public static String alterarAmigo (Amigo amigo) {
    String sql = "UPDATE Amigos SET Apelido = ?, Cidade = ?, Email = ?, Sexo = ?, "
        + " EstadoCivil = ?, Whatsapp = ? WHERE Nome = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, amigo.getApelido());
        comando.setString(2, amigo.getCidade());
        comando.setString(3, amigo.getEmail());
        comando.setInt(4, amigo.getSexo().ordinal());
        comando.setInt(5, amigo.getEstadoCivil().ordinal());
        comando.setString(6, amigo.getWhatsapp());
        comando.setString(7, amigo.getNome());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace();
        return "Erro na Alteração do Amigo no BD";
    }
}
```

5.4 - O Tratamento de Eventos para Remover um Amigo

Na implementação do tratador de eventos `removerAmigo` associado ao botão de comando `Remover`, a chave do amigo (`nome`) é lida no campo de texto rotulado por `Nome`:

```
private void removerAmigo(java.awt.event.ActionEvent evt) {
    Amigo visão = (Amigo) amigos_cadastradosComboBox.getSelectedItem ();
    String mensagem_erro = null;
    if (visão != null) mensagem_erro = controlador.removerAmigo(visão.getNome());
    else mensagem_erro = "Nenhum amigo selecionado";
    if (mensagem_erro == null) {
        amigos_cadastradosComboBox.removeItem(visão);
        limparCampos();
    } else informarErro (mensagem_erro);
}
```

A seguir, a ilustração do método `removerAmigo` da classe `ControladorCadastroAmigos`, do pacote `controles`:

```
public String removerAmigo (String nome) {
    Amigo amigol = Amigo.buscarAmigo(nome);
    if (amigol != null) return Amigo.removerAmigo(nome);
    else return "Nome de amigo não cadastrado";
}
```

A seguir a ilustração do método estático `removerAmigo` da entidade `Amigo`, do pacote `entidades`:

```
public static String removerAmigo (String nome) {
    String sql = "DELETE FROM Amigos WHERE Nome = ?";
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
        return "Erro na Remoção do Amigo no BD";
    }
}
```

5.5 - O Tratamento de Eventos para Limpar Campos

O tratador de eventos `limparCampos` é ilustrado a seguir.

```
private void limparCampos(java.awt.event.ActionEvent evt) {
    nomeTextField.setText("");
    apelidoTextField.setText("");
    cidadeTextField.setText("");
    emailTextField.setText("");
    sexoTextField.setText("");
    estado_civilTextField.setText("");
    whatsappTextField.setText("");
}
```

6 - Ilustrando uma Inserção na JanelaCadastroAtores

A classe [Ator](#), do pacote [entidades](#), é definida com os seguintes enumerados e atributos:

```
public class Ator {  
  
    public enum Sexo { feminino, masculino };  
  
    private String nome;  
    private int ano_nascimento;  
    private Sexo sexo;  
    private boolean ganhador_oscar;  
}
```

Para a [JanelaCadastroAtores](#) é ilustrada somente a interface com um objeto preenchido, dado que a sua implementação é equivalente à implementação da [JanelaCadastroAmigos](#).

A interface da JanelaCadastroAtores apresenta o seguinte layout:

- Um menu suspenso "Atores Cadastrados" com o valor "Kate Winslet [1975]".
- Um campo de texto "Nome" com o valor "Kate Winslet".
- Um campo de texto "Ano de Nascimento" com o valor "1975".
- Um campo de texto "Sexo" com o valor "F".
- Um campo de texto "Ganhador de Oscar" com o valor "S".
- Quatro botões de ação: "Inserir", "Consultar", "Alterar" e "Remover".
- Um botão "Limpar" localizado à direita dos outros botões.